МИНИСТЕРСТВО ОБРАЗОВАНИЯ И НАУКИ РЕСПУБЛИКИ КРЫМ

ГБОУВО РК «Крымский инженерно-педагогический университет имени Февзи Якубова»

ОТЧЁТ

Групповой проект. Команда 7

Сортировка в современных языках программирования

Дисциплина: Алгоритмы и структуры данных

Преподаватель: Меметов М. Р.

Симферополь, 2026

1. Состав и цель

Лидер: Подъячий Глеб Русланович. Участники: Керимов Андрей Эдуардович, Мамбетов Родион Рудемович, Черакаев Антон Александрович, Кулиш Артём Евгеньевич.

Цель: выяснить, какой алгоритм спрятан во встроенной сортировке Python, Java, C#, JavaScript и C++, и на одних и тех же массивах сравнить время и устойчивость.

2. Алгоритмы

Python 3.14, list.sort. Устойчивая адаптивная сортировка слиянием семейства Timsort. С версии 3.11 порядок слияния серий выбирает Powersort. На полностью упорядоченном массиве время близко к линейному.

Java 17. Arrays.sort для int, long и чисел с плавающей точкой: Dual-Pivot Quicksort, не устойчивый. Arrays.sort для объектов: устойчивый Timsort.

C#, Array.Sort. Introsort: быстрая сортировка, вставки на кусках не длиннее 16 и пирамида при глубокой рекурсии. Не устойчивый. Устойчивый вариант: LINQ OrderBy, при равенстве сравнивается исходный номер.

JavaScript. ES2019 требует устойчивость, алгоритм не назначает. Node.js 24 работает на V8, там Timsort. Без функции сравнения числа сортируются как строки.

C++, MSVC. std::sort: introsort, на месте, не устойчивый. std::stable_sort: слияние, устойчивый, нужна дополнительная память.

3. Сложность

Реализация	Алгоритм	Время	Память	Устойчивость
Python list.sort	Timsort, Powersort	O(n log n), на серии O(n)	O(n)	да
Java int[]	Dual-Pivot Quicksort	O(n log n)	O(log n)	нет
Java Integer[]	Timsort	O(n log n), на серии O(n)	O(n)	да
C# Array.Sort	introsort	O(n log n)	O(log n)	нет
JavaScript, V8	Timsort	O(n log n), на серии O(n)	O(n)	да
C++ std::sort	introsort	O(n log n)	O(log n)	нет
C++ stable_sort	слияние	O(n log n)	O(n)	да

Для случайных данных все семь вызовов имеют порядок n log n. При росте n в 10 раз время растёт примерно в 12 раз. На уже упорядоченных данных Timsort и Dual-Pivot, который замечает готовые серии, уходят к линейному проходу. Introsort тоже быстро заканчивает, но запасной пирамидой держит худший

случай в границе O(n log n). Устойчивые слияния просят дополнительную память O(n). Introsort и быстрая сортировка живут почти на месте, память O(log n) на стек.

4. Как измеряли

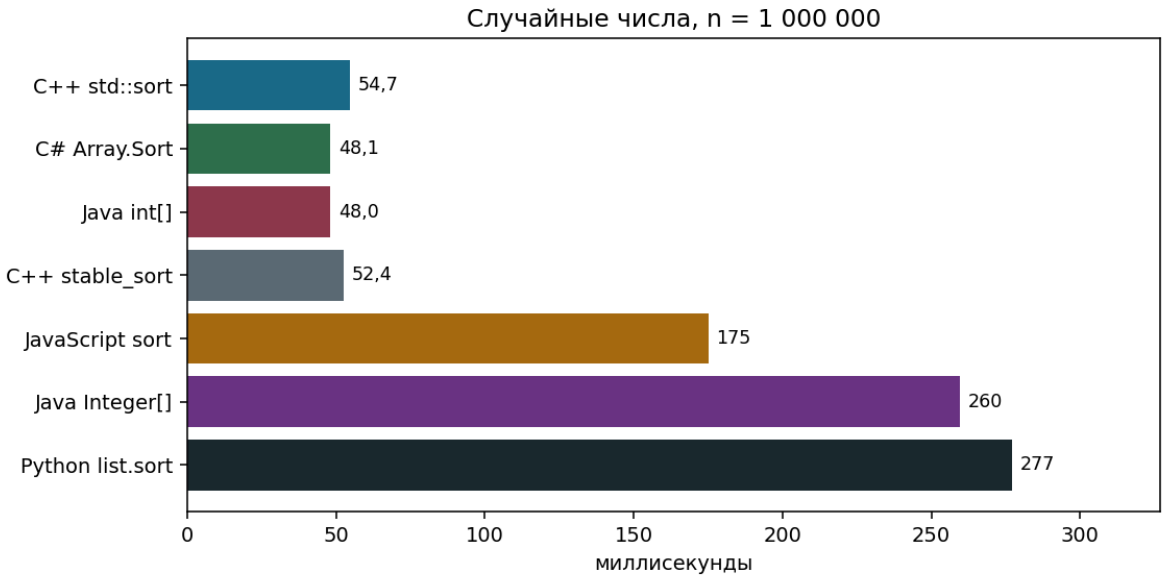
Один генератор с зерном 7 построил пять видов массивов: случайный, по возрастанию, по убыванию, из 10 разных чисел и почти упорядоченный. Размеры: 10 тысяч, 100 тысяч, 1 миллион. Все языки читали один и тот же файл. Первая попытка отбрасывалась, дальше бралась медиана пяти замеров. В время не входит чтение файла.

Устойчивость: 400 пар «значение, исходный номер» сортируются только по значению. Если номера внутри равного значения идут по порядку, алгоритм устойчивый.

5. Результаты, n = 1 000 000, миллисекунды

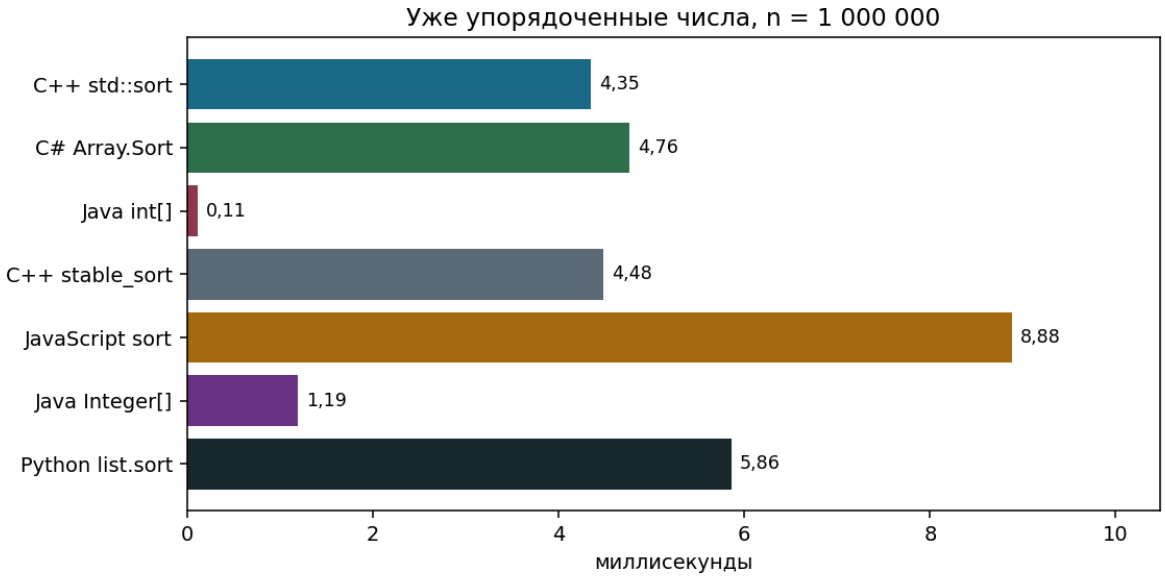
Реализация	Случайные	По возрастанию	По убыванию	Мало разных	Почти упорядочен
C++ std::sort	54,7	4,35	5,65	8,86	4,92
C# Array.Sort	48,1	4,76	9,23	17,3	4,92
Java int[]	48,0	0,11	0,64	9,05	4,44
C++ stable_sort	52,4	4,48	8,04	17,3	4,36
JavaScript sort	175	8,88	8,94	84,3	20,3
Java Integer[]	260	1,19	6,65	130	4,04
Python list.sort	277	5,86	6,91	96,2	14,5

Рисунок 1. Случайный массив из миллиона чисел. Три быстрые реализации для голых чисел стоят рядом, три реализации с объектами или с динамическим языком заметно правее.



На случайном миллионе Java для int, C# и C++ std::sort близки: 48, 48 и 55 мс. Python тратит 277 мс, Java для объектов 260 мс, JavaScript 175 мс. Это цена сравнения объектов и устройства языка, а не приговор Timsort. От 100 тысяч элементов к миллиону время C++ выросло примерно в 12 раз, время Python примерно в 14 раз. Формула n log n при росте n в десять раз как раз обещает около двенадцати.

Рисунок 2. Тот же миллион, но числа уже стоят по возрастанию. Шкала другая: здесь счёт идёт на единицы миллисекунд, а не на сотни.



На упорядоченном миллионе Java для int показывает 0,11 мс против 48 мс на случайных числах. Python показывает 5,86 мс против 277 мс. Готовый порядок адаптивные реализации используют. Массив из десяти разных значений, наоборот, тяжелее для устойчивых сортировок объектов: Python тратит 96 мс, Java для объектов 130 мс, а std::sort остаётся на 8,86 мс.

Проверка устойчивости сделана отдельно от таблицы времени, на четырёхстах парах. Python, JavaScript, объекты Java и std::stable_sort сохранили исходные номера внутри равного значения. C# Array.Sort и C++ std::sort их переставили. Короткий пример из десяти элементов для этой проверки не годится: introsort на таком куске часто доделывает работу вставками и случайно сохраняет порядок.

6. Кто что делает на защите

Подъячий Глеб Русланович, лидер, открывает тему и рассказывает Python. Керимов Андрей Эдуардович показывает, что в Java два алгоритма. Мамбетов Родион Рудемович отделяет Array.Sort от устойчивого OrderBy. Черакаев Антон Александрович сравнивает std::sort и std::stable_sort. Кулиш Артём Евгеньевич показывает JavaScript и ловушку сортировки чисел как строк. Каждый говорит не меньше одной минуты. Демонстрация идёт по файлу demo.html.

7. Выводы

Выбор зависит от двух вещей: важен ли порядок равных элементов и есть ли в данных готовый порядок. Числа и скорость: std::sort, Array.Sort или Arrays.sort для int. Прежний порядок равных записей: list.sort, сортировка объектов в Java, Array.prototype.sort с числовым компаратором, std::stable_sort или OrderBy. В JavaScript вызов без функции сравнения для чисел является ошибкой.

Источники: CPython listsort.txt и Powersort с версии 3.11; документация Java SE 17 Arrays; Microsoft Array.Sort; блог V8 «Getting things sorted»; MDN, устойчивость с ES2019; стандарт C++ для std::sort и std::stable_sort.